CryptoCopilot

The Complete Project Guide

A polyglot, paper-only crypto trading assistant — ML + technical analysis + fundamentals + cited RAG chat, fused into one explainable opinion.

Complete technical guide — ML, Backend, Frontend, and AI

Decision-support, not financial advice · Paper trading only

Version 1.0 · June 2026

Contents

1. What CryptoCopilot Is

CryptoCopilot is a personal assistant for someone who is new to crypto and wants help making sense of the market. It answers one focused question for a single retail trader:

> "Given everything happening in the market and the news right now, what should I think about these coins — and if I make a trade, how would it perform?"

It answers that question by looking at each of **10 coins** from **four different angles** at the same time, and then combining them into one clear, explainable opinion:

- **Machine Learning (ML)** — a model that estimates the probability the price goes **up, down, or stays flat** over the next 24 hours.
- **Technical Analysis (TA)** — a rule-based reading of classic chart indicators (Ichimoku, RSI, MACD, Bollinger, ATR).
- **Fundamentals** — a health check from on-chain activity and community / developer data.
- **News + RAG chat** — a "Researcher" that answers questions using only real, cited sources (news, on-chain data, and a curated knowledge base).

A component called the **Analyst** fuses these four views into a single verdict — a **direction**, a **conviction** level, and an **agreement** score. The user can then act by placing a **paper trade** (practice trade with fake money). Every number is traceable, every trade is logged, and a disclaimer is shown on every page.

Honest scope — engineering first

This is deliberately a **portfolio / learning project**. The goal is **not** to beat the market (no small model can reliably do that). The goal is to demonstrate clean, production-grade engineering across three languages and several disciplines: data pipelines, machine learning, a Java backend, retrieval-augmented AI, and a modern web frontend. The results are reported **honestly** — for example, the ML model's score is modest on purpose, and we explain exactly why.

> ⚠️ Decision-support, not financial advice. Paper trading only — no real money, ever. Crypto only; no shorts, no leverage.

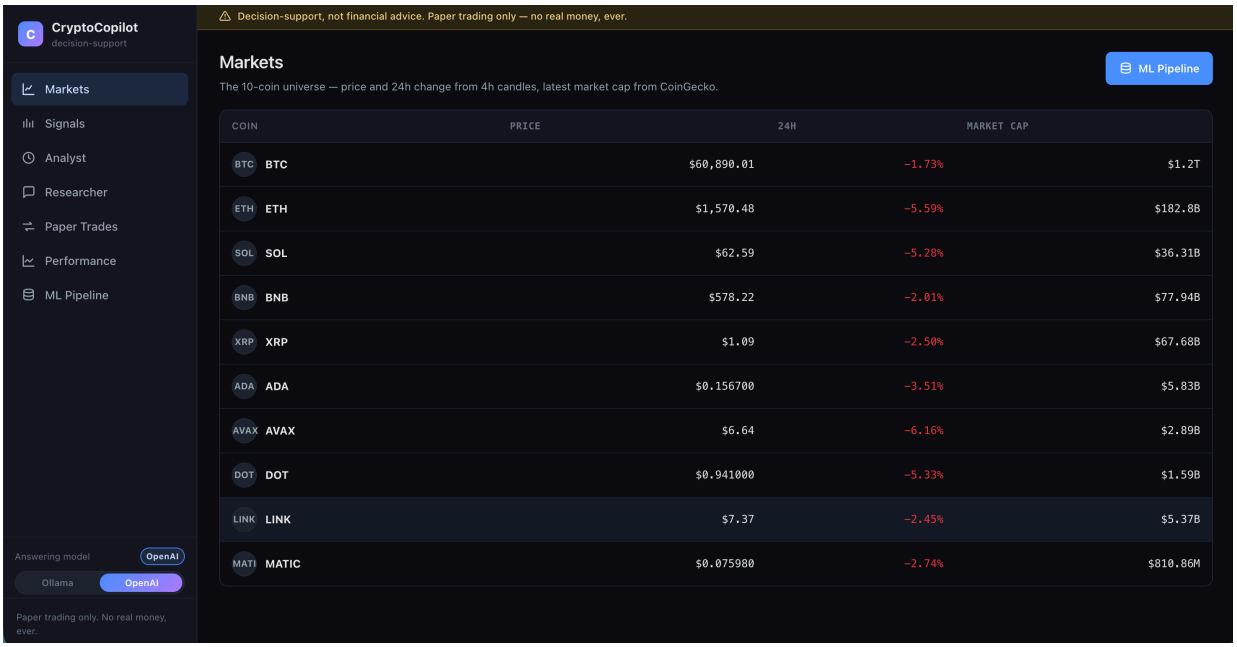

Figure 1 — the Markets page: the 10-coin universe with live price and 24h change. The sidebar (left) lists every page; the "Answering model" toggle (bottom-left) switches the AI between local Ollama and OpenAI.

2. The Big Picture: A Polyglot Architecture

"Polyglot" means the system is built from **more than one programming language**, each used where it is strongest. CryptoCopilot runs as **five Docker containers** around **one shared database**:

Container	Language / tech	Job
db	Postgres 16 + pgvector	The single source of truth — stores all data and the AI vector index.
ml	Python (batch worker)	Wakes on a schedule, fetches data, runs the ML model, writes results, sleeps.
ml-api	Python (FastAPI)	Lets you trigger ingest / train / predict on demand (same code as ml).
backend	Java + Spring Boot	The brain of the app: REST API, technical analysis, RAG chat, Analyst, paper trading.
frontend	React + nginx	The website you see in the browser — a thin client over the backend.

The text diagram below shows how they connect. The most important idea is the arrow into the database from two languages at once:

```
browser  →  frontend (React + nginx)
                 │  REST / JSON  (same origin; nginx proxies /api)
                 ▼
            backend (Spring Boot)  ── MODULAR MONOLITH (ta4j · Spring AI · trading · analyst)
                 │ JDBC            │ HTTP  /api/ml/* → ingest·train·predict
                 ▼                 ▼
 (writes preds)  db (Postgres + pgvector)     ml-api (Python · FastAPI)
      ▲                                          │  (same code, shared model dir)
 ml (Python batch worker) ──────────────────────┘  writes predictions / raw data
```

Why this shape (the design decisions)

The database is the boundary between the two languages. Python writes its tables; Java only reads them. There is no direct call from Java into Python's model, no shared model file, no network "remote procedure call" between them. They cooperate purely through SQL tables. This is simple, robust, and easy to reason about — even if the Python service is switched off, the app still serves the last results it wrote.

The backend is a "modular monolith", not microservices. It is one Java application with clean internal modules (technical analysis, RAG, trading, analyst). Splitting it into many tiny services would add a lot of operational pain (service discovery, network calls, distributed transactions) to solve scaling problems a single-user project simply does not have.

ML is a batch job, not a live web service. A 24-hour price direction changes slowly, so the `ml` container wakes on a schedule, does its work, writes to the database, and sleeps. The sibling `ml-api` container puts a thin web layer in front of the *same* code so those jobs can also be launched on demand from the UI's "ML Pipeline" page.

Table ownership — exactly one writer per table

To keep the shared database clean, **every table has exactly one writer**. Java never writes Python's tables; Python never writes Java's.

Owner	Writes these tables	Reads
ml (Python)	ohlcv, market_meta, news, onchain, fundamentals, predictions, prediction_drivers	its own tables
backend (Java)	account_state, positions, trades, orders, the Spring-AI vector_store	all of ml's tables (read-only)
frontend (React)	—	backend REST only

3. The Data Layer — Coins, Sources, and the Database

The 10 coins, and why these

The universe is the **10 largest, most liquid crypto-assets** — the ones with the deepest market history, the most news coverage, and the most on-chain data. That depth is exactly what a data and ML project needs. They also cover several *different* designs, which makes the project more interesting than 10 near-identical coins:

Symbol	Name	What it is / why it is in the set
BTC	Bitcoin	The original cryptocurrency; Proof-of-Work; the market benchmark and the only coin with a rich daily on-chain series here.
ETH	Ethereum	The largest smart-contract platform; Proof-of-Stake; runs most of DeFi and NFTs.
SOL	Solana	A high-throughput Proof-of-Stake chain; fast and cheap; a major Ethereum competitor.
BNB	BNB	The exchange/utility token of the BNB ecosystem; large and liquid.
XRP	XRP	Focused on fast, cheap cross-border payments; a distinctive consensus design.
ADA	Cardano	A research-driven Proof-of-Stake platform.
AVAX	Avalanche	A fast Proof-of-Stake platform with a multi-chain ("subnet") design.

DOT	Polkadot	An interoperability network connecting many chains (“parachains”).
LINK	Chainlink	The leading oracle network — it feeds real-world data into smart contracts.
MATIC/POL	Polygon	An Ethereum scaling network. (MATIC was rebranded POL in late 2024; the loader stitches both under MATIC.)

Data sources — all public and free, with URLs

Every source is public and free. A core rule (“no single point of failure”) means that if any source goes down or paid, the pipeline **logs the problem and skips it — it never crashes**.

Source	Used for	URL	Auth
Binance public API	OHLCV candles (1h / 4h / 1d, ~2 years)	binance.com/api	none
CoinGecko Demo	Market cap, supply, community + developer + market data (all 10)	coingecko.com/en/api	free key (10k/mo, 30/min)
CoinDesk RSS	News headlines (180-day window)	coindesk.com/arc/outboundfeeds/rss/	none
Cointelegraph RSS	News headlines	cointelegraph.com/rss	none
Decrypt RSS	News headlines	decrypt.co/feed	none
The Block RSS	News headlines	theblock.co/rss.xml	none
Bitcoin Magazine RSS	News headlines	bitcoinmagazine.com/.rss/full/	none
Blockchain.com Charts	BTC on-chain metrics	blockchain.com/charts	none
Etherscan	ETH on-chain metrics	etherscan.io/apis	free key (5/sec, 100k/day)
Curated Knowledge Base	Coin mechanism / tokenomics (authored in-repo)	—	—

What “OHLCV” means: for each time period (a “candle”), we store the **O**pen, **H**igh, **L**ow, and **C**lose price plus the traded **V**olume. This is the raw material for both the ML features and the technical-analysis indicators.

The first full ingestion loaded **231,082 rows**: `ohlcv` ≈ 226,200 (10 coins × 3 timeframes × ~2 years), `market_meta` 3,660, `news` 124 (a rolling window), `onchain` 1,088 (BTC + ETH), and `fundamentals` 10 (one latest snapshot per coin).

The database model

All data lives in **Postgres** (a relational database). Two roughly separate groups of tables — Python-owned (data + ML) and Java-owned (the paper-trading account) — share one schema defined in `db/init.sql`. The diagram below shows every table and its columns.

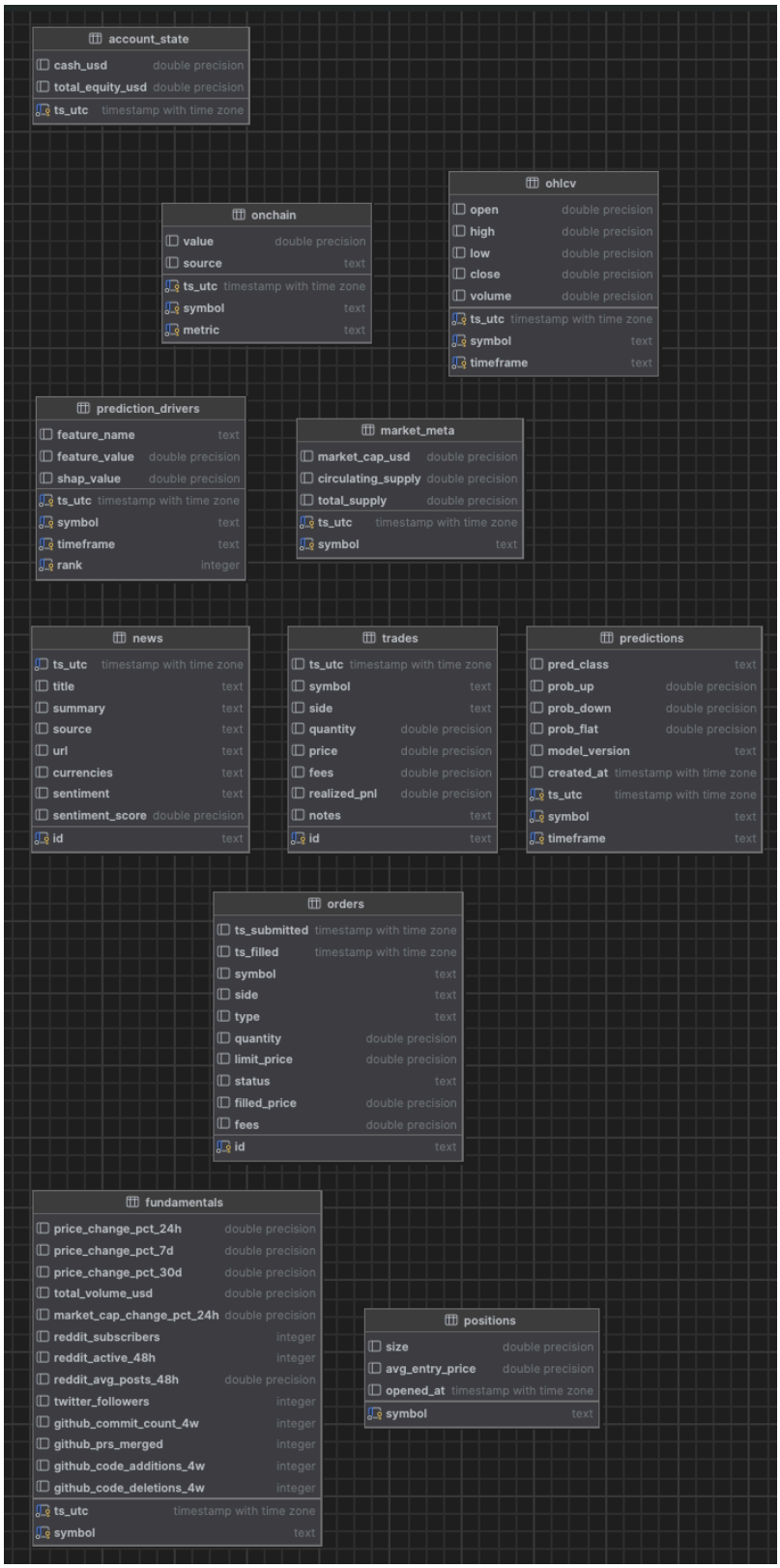

Figure 2 — the database schema. Python-owned tables (top): ohlcv, market_meta, news, onchain, fundamentals, predictions, prediction_drivers. Java-owned (bottom): account_state, positions, trades, orders. Spring AI adds its own vector_store.

Table	Owner	What each row holds

ohlcv	ml	One price candle: timestamp, symbol, timeframe, open/high/low/close, volume.
market_meta	ml	Market cap and circulating / total supply at a point in time.
news	ml	A news item: title, summary, source, url, tagged coins, and a sentiment label + score.
onchain	ml	An on-chain metric value (e.g. active addresses) for a symbol at a time, with its source.
fundamentals	ml	Community + developer + market snapshot: 24h/7d/30d change, volume, Reddit/Twitter/GitHub activity.
predictions	ml	The model's latest forecast per coin: predicted class + probability of up/down/flat.
prediction_drivers	ml	The top-3 SHAP reasons behind each prediction (feature name, value, contribution).
account_state	backend	A snapshot of the paper account: cash and total equity over time (the equity curve).
positions	backend	Currently held coins: size and average entry price.
trades	backend	Every executed buy/sell with quantity, price, fees, and realized profit/loss.
orders	backend	Order tickets: market/limit, status (pending/filled/cancelled), fill price.
vector_store	Spring AI	The AI search index: text chunks + their embedding vectors (768 numbers each).

4. The Machine Learning Service (Python)

This is the heart of the project. The story, in one line: **turn ~2 years of raw price candles into a calibrated, explainable probability that each coin will move up, down, or stay flat over the next 24 hours.** Let us walk through every step as if from scratch.

Step 1 — Ingestion (getting the data)

Python is excellent at data work, so all data fetching lives here. The libraries used:

- **ccxt** — a unified library for crypto exchanges; we use it to download Binance OHLCV candles.
- **requests** — plain HTTP calls to CoinGecko, Blockchain.com, and Etherscan.
- **feedparser** — reads the RSS news feeds.
- **vaderSentiment** — a lightweight, rule-based tool that labels each news headline as positive / negative / neutral, locally and for free (explained in the glossary).
- **pandas / numpy** — the standard tools for tables of numbers and math.
- **SQLAlchemy + psycopg2** — write the results into Postgres. **APScheduler** — the timer that runs ingestion daily.

Step 2 — Feature engineering (turning candles into signals)

A machine-learning model cannot learn from raw prices directly; we must compute **features** — informative numbers that describe the current market state. CryptoCopilot computes **46 features** per coin per time step, all **backward-looking only** (they never peek at the future), including:

- **Returns** over 1h, 4h, 24h, and 7d (how much price changed recently).
- **RSI** at 7/14/21 periods, **MACD** + its crossover, **Stochastic**, **ADX** (trend strength).
- **Bollinger %B** and bandwidth, **ATR%** (volatility), realised volatility (24h / 7d).
- **Volume z-score** (is volume unusually high?), and **moving-average ratios**.

- **Ichimoku** components, written from scratch (Tenkan, Kijun, Senkou A/B, cloud flags and distances), with a leakage-safe time shift.
- Calendar features (e.g. day of week) and a one-hot code for the coin's identity.

Feature engineering stays **internal to Python** (cached as Parquet files); only the final predictions cross the database boundary into Java. The small duplication — Java recomputes its own indicators with ta4j — is intentional and keeps the two languages cleanly separated.

Step 3 — The target and the splits (defining the question)

We turn the problem into **3-class classification**. The **target** (`y_24h_3class`) labels each moment by what the price did over the *next* 24 hours:

- **UP** if it rose more than +2%, **DOWN** if it fell more than −2%, **FLAT** otherwise.

Why ±2%? A smaller band (say ±1%) mostly captures random noise; ±2% targets a move big enough to matter. The data is then split **strictly by time** — train on the past, validate on a later slice, test on the most recent slice — with a **24-hour "embargo"** gap between splits so that no information leaks across the boundary. Respecting time order is essential: shuffling rows would let the model "see the future," which would make the score look great but be meaningless.

Split	Rows	Window	DOWN / FLAT / UP balance
train	20,961	2024-06-15 → 2025-05-30	0.268 / 0.466 / 0.265
val	5,460	2025-06-01 → 2025-08-30	0.246 / 0.473 / 0.282
test	16,290	2025-09-01 → 2026-05-30	0.261 / 0.537 / 0.202

Step 4 — XGBoost (the model), explained

The main model is **XGBoost**. To understand it, build up three ideas:

- **A decision tree** is a flowchart of yes/no questions about the features (e.g. "is RSI below 30?") that ends in a prediction. One tree alone is weak and tends to overfit.
- **Boosting** trains many small trees **in sequence**, where each new tree focuses on fixing the mistakes the previous trees made. Adding up many weak trees produces one strong model.
- **Gradient boosting** does this "fixing" using the mathematics of gradients (the direction that reduces error fastest). **XGBoost** is a fast, regularized, industrial-strength implementation of gradient-boosted trees.

Why XGBoost for this problem? Our features are **tabular** (rows and columns of numbers), they interact in non-linear ways, and they are noisy. Gradient-boosted trees are the proven best-in-class choice for exactly this kind of data — they capture interactions automatically, handle mixed scales, and resist overfitting through regularization. We configure it for multi-class output (`multi:softprob`, which returns a probability for each of the 3 classes).

Tuning with Optuna. A model has "hyper-parameters" (knobs like tree depth and learning rate). **Optuna** searches these automatically — it ran **40 trials**, each time training on `train` and scoring on `val`, keeping the best. The winner: shallow trees (`max_depth=4`), a slow learning rate (`0.029`), and strong regularization — a deliberately cautious model that generalizes rather than memorizes. We also fit a simple **Logistic Regression** as a baseline; XGBoost beats it (macro F1 0.375 vs 0.292), which confirms the extra complexity is earning its keep.

Step 5 — Isotonic calibration (making probabilities honest)

A model can rank cases well but still output **mis-scaled** probabilities — for example, it might say "80%" for events that actually happen only 60% of the time. **Calibration** fixes the *numbers* so that, when the model says 70%, the event truly happens about 70% of the time.

Isotonic regression is a flexible, non-parametric calibration method: it learns any monotonic (always-increasing) mapping from raw score to true frequency. We fit it **on the validation set only** (never on training or test data — otherwise it would cheat). The result is **honest probabilities**, which matter here because the Analyst and the UI show confidence numbers to the user — those numbers must mean what they say. We verify calibration with the **Brier score** (0.606; lower is better).

Step 6 — SHAP (explaining each prediction)

A model that just says "DOWN" is a black box. **SHAP** opens it. SHAP is based on **Shapley values**, an idea borrowed from game theory: treat each feature as a "player" in a team and fairly share out the credit (or blame) for the final prediction among them. For every prediction, SHAP tells us **how much each feature pushed the result up or down**.

We use SHAP's `TreeExplainer` (fast and exact for tree models) and store the **top-3 drivers** for each coin in the `prediction_drivers` table — so the UI can show *why*, e.g. "the biggest reasons this is DOWN were day-of-week, ADX, and the MACD signal." (The coin-identity feature is excluded from drivers, so the reasons describe the *market state*, not merely "this coin is BTC.")

Step 7 — Evaluation: how good is it, really?

ROC-AUC — the headline metric, explained in full

ROC-AUC (Area Under the Receiver-Operating-Characteristic Curve) measures how well a model **ranks** positive cases above negative ones. Imagine picking one real "UP" case and one real "not-UP" case at random: **AUC is the probability the model gives the true UP case the higher score.** Read it like this:

- **AUC = 0.5** → the model is no better than a coin flip (no skill).
- **AUC = 1.0** → perfect ranking.
- **AUC = 0.578 (ours)** → a small but real edge above random — the model has learned *something*, just not a lot.

Why is AUC the fair headline here? Because it does not depend on a single decision threshold and is not fooled by imbalanced classes (FLAT is the most common outcome). A naïve model that always says "FLAT" could look accurate but would score AUC ≈ 0.5; ours scores **0.578**, which is exactly in the honest, expected band for predicting short-term crypto direction from price data alone. **Anything much above ~0.65 here would be a red flag for data leakage**, not genuine skill.

The other metrics

Metric	Our value	Target	Meaning
macro ROC-AUC	0.578	0.55–0.62	Ranking skill (above 0.5 = real edge). PASS.
multiclass Brier	0.606	≤ 0.65	Probability accuracy (lower is better). PASS.
macro F1	0.375	≥ 0.40	Balance of precision & recall across classes. Just short — see below.

baseline LogReg F1	0.292	—	Simple baseline; XGBoost clearly beats it.

Why macro F1 is 0.375 (and why that is honest, not a bug). The hardest class is **UP** — predicting a >2% rally in the next 24 hours from price data alone, in a calm 2025–26 market, is near the noise floor. The team investigated all the usual suspects and ruled them out: it is **not leakage** (AUC sits in the expected band; a leakage test passes), it is **not the decision rule** (even a hindsight-optimal rule caps at the same 0.375), and tightening the target made it worse. The real limit is **data volume** — about 2 years of history. More history is the lever most likely to push past 0.40. This is reported as the deliberate, data-limited result it is.

Figure 3 — one coin's signal. The model's class (DOWN, 30.2%) and the probability bar, then the top-3 SHAP drivers, then the deterministic ta4j technical verdict with the exact rules that fired.

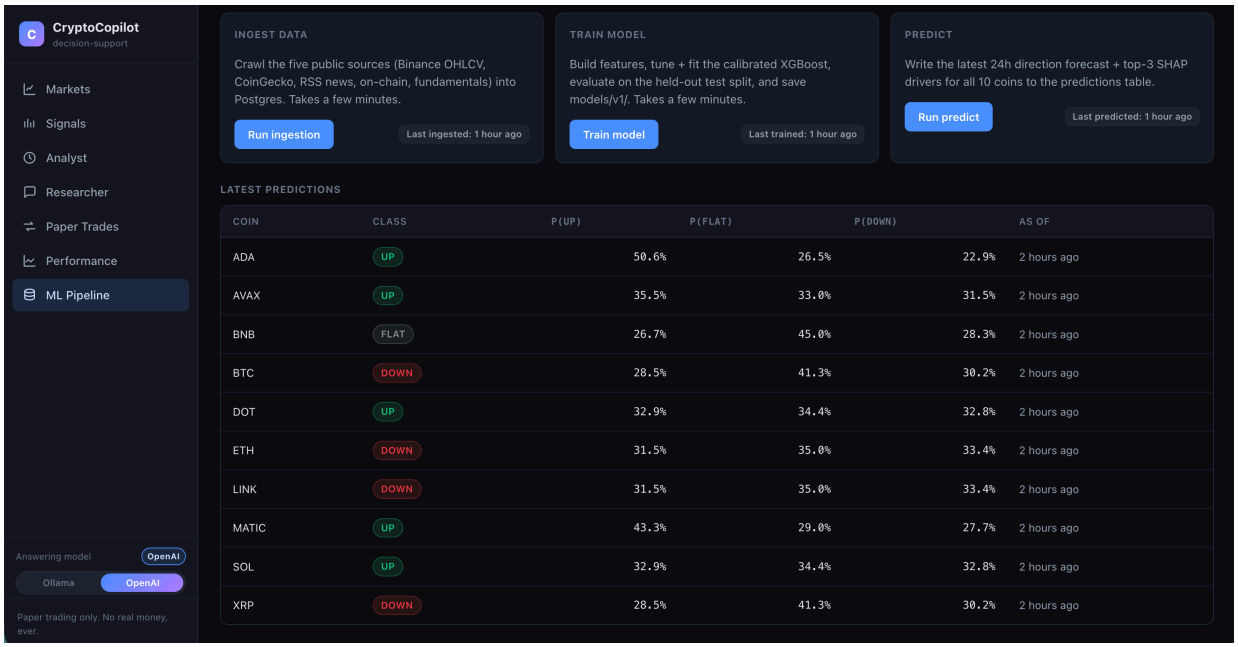

Figure 4 — the ML Pipeline page. The three jobs (ingest → train → predict) can be run on demand; below, the latest per-coin probabilities for UP / FLAT / DOWN.

5. The Backend Service (Java / Spring Boot)

The backend is the application "brain." It is written in **Java 21** with **Spring Boot**, the industry-standard framework for building reliable server applications (it wires everything together, manages the database connection, and serves the REST API). It is one **modular monolith** with internal modules for data, technical analysis, RAG, the Analyst, and trading.

Spring AI — what it is and how we use it

Spring AI is Spring's official library for adding AI features to a Java app. It gives a clean, provider-independent way to call a chat model (an LLM) and an embedding model, and to talk to a vector database. We use it for **two** things: the **Researcher (RAG) chat** and the Analyst's **LLM-phrased summary**. Because Spring AI hides the provider behind one interface, we can switch between **local Ollama** and **OpenAI** by changing configuration — the application code does not change. (Details in the AI section.)

ta4j — technical-analysis indicators in Java

ta4j ("Technical Analysis for Java") is an open-source library of trading indicators. The backend uses it to compute its **own** technical verdict directly from the raw `ohlcv` candles — completely independent of the Python ML model. The indicators implemented:

Indicator	What it measures	How CryptoCopilot uses it
Ichimoku Cloud	Trend, support/resistance, momentum (a full system)	The centerpiece: is price above/below the cloud? Is the cloud bullish? (9/26/52 settings)
RSI (14)	Momentum: overbought (>70) / oversold (<30)	Hedges the verdict — an oversold reading can soften a bearish score.

MACD (12,26,9)	Trend & momentum via moving-average differences	A rising positive histogram adds a bullish point.
Bollinger %B (20,2)	Where price sits within its volatility bands	Flags stretched / mean-reversion conditions.
ATR	Volatility (average true range)	Context for how large recent moves are.

These rules combine into a numeric **score** that maps to a **direction** (BULLISH / NEUTRAL / BEARISH) and a **confidence** (STRONG / MODERATE / WEAK), plus the **list of exactly which rules fired** — so the verdict is fully transparent and reproducible. A care point worth noting: the Ichimoku "displacement" is applied in a **leakage-safe** way that mirrors the Python side, so the indicator never accidentally uses future candles.

Java libraries used, and why

Library	Purpose
Spring Boot (web, data-jpa)	The application framework: REST endpoints, dependency injection, configuration.
Spring Data JPA + Hibernate	Maps Java objects to database tables; `ddl-auto: validate` checks the code matches the schema.
PostgreSQL JDBC driver	Connects Java to the Postgres database.
Spring AI (+ pgvector + Ollama/OpenAI starters)	The RAG chat and the Analyst summary; the vector store.
ta4j	Technical-analysis indicators (the TA verdict).
springdoc-openapi	Auto-generates interactive API docs (Swagger UI).
JUnit 5 + Mockito	Automated tests (70 offline tests on the backend).

The whole API is self-documented with **Swagger UI** (an interactive web page listing every endpoint), generated automatically by springdoc.

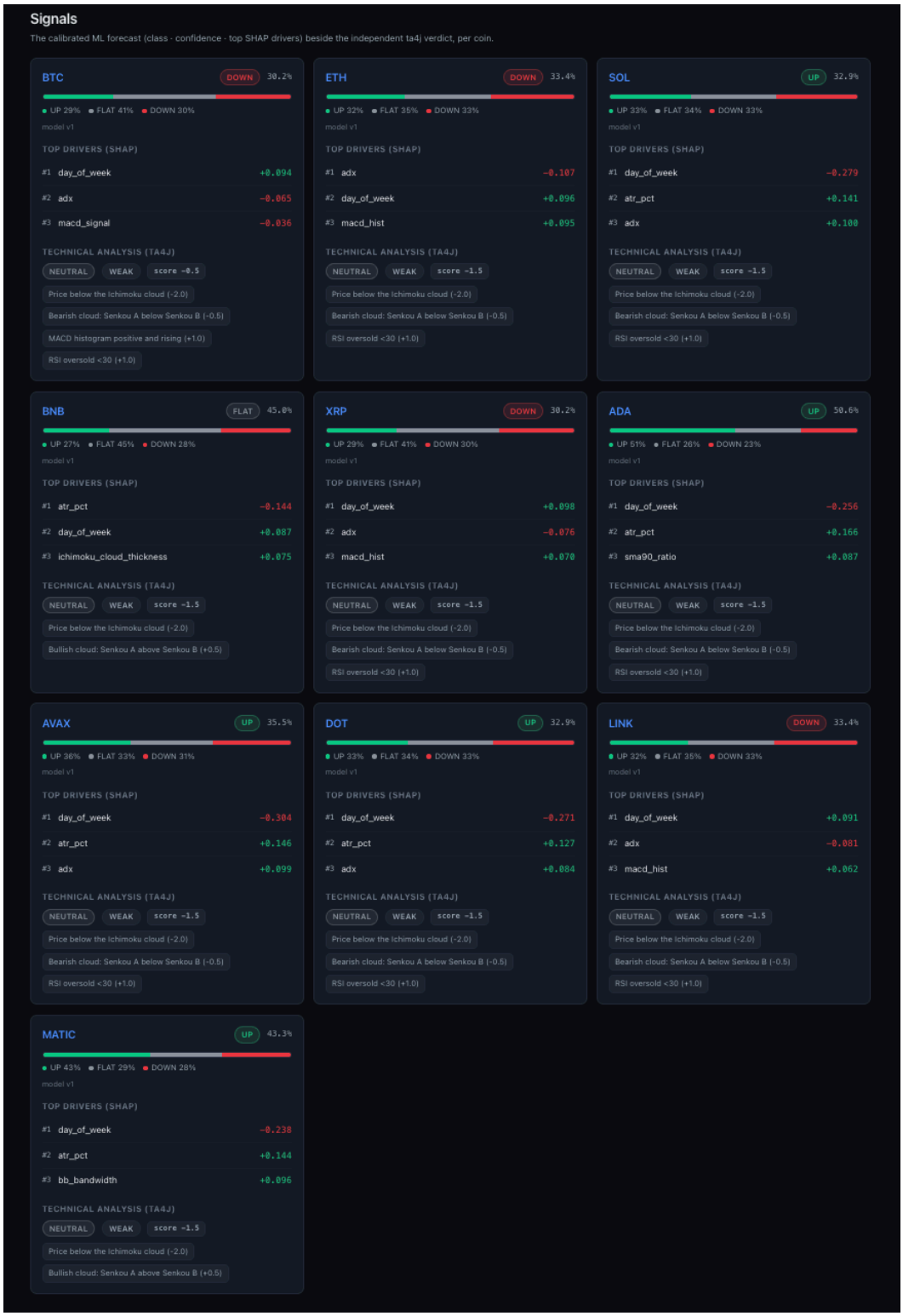

Figure 5 — the Signals page: for all 10 coins, the ML class + confidence, the probability bar, the top-3 SHAP drivers, and the ta4j technical verdict side by side.

6. The AI Layer — Embeddings & RAG (the Researcher)

What is an embedding? (from scratch)

Computers do not understand words; they understand numbers. An **embedding** is a way to turn a piece of text into a **list of numbers** (a "vector") that captures its **meaning**. The magic property: texts with **similar meaning** get **similar vectors**, even if they use different words. "Bitcoin mining difficulty rose" and "BTC hashrate adjustment increased" land close together; "I love pizza" lands far away.

Each of our embeddings is a list of **768 numbers**. To find text relevant to a question, we embed the question and then look for the stored chunks whose vectors are **closest** to it. "Closeness" is measured by **cosine similarity** — essentially the angle between two vectors; a small angle means similar meaning.

What is RAG, and why use it?

RAG stands for **Retrieval-Augmented Generation**. A large language model (LLM) on its own will happily **make things up** ("hallucinate") and has no knowledge of *your* private, up-to-date data. RAG fixes both problems in three steps:

1. **Retrieve** — embed the user's question and fetch the most relevant real chunks from our own data (news, on-chain summaries, fundamentals, and the knowledge base).
2. **Augment** — paste those chunks into the prompt as the *only* allowed source material.
3. **Generate** — ask the LLM to answer **using only those chunks**, and to **cite** them with markers like [1], [2].

The result is an answer that is **grounded in real sources and cited**, instead of confident fiction. CryptoCopilot's Researcher is strict about this: if the sources do not contain the answer, it **refuses** with a fixed phrase rather than guessing, and it **always refuses to give trading advice**. On a 20-question evaluation it reached **recall@8 = 0.90** (for 90% of questions, a correct chunk was in the top 8 retrieved) with a **100% citation rate**.

Why pgvector?

To retrieve by meaning we need a database that can store vectors and find the nearest ones quickly. **pgvector** is an extension that adds a `vector` column type and fast similarity search **to Postgres** — the database we already use. That is the key benefit: **no extra database to run.** Our embeddings and our relational data live in **one** Postgres instance. Spring AI automatically creates and manages the `vector_store` table (768-dim, with an HNSW index for fast approximate nearest-neighbour search using cosine distance).

Ollama vs OpenAI — why both exist (the toggle)

CryptoCopilot can use **two** AI providers, chosen at runtime with a toggle in the sidebar:

- **Ollama (the default)** runs a model **locally on your own computer** — model `llama3.2:3b`. It is **free, private, and needs no API key**. The trade-off is that a small local model gives shorter, simpler answers.
- **OpenAI (optional)** uses the **gpt-4o-mini** model in the cloud. It gives noticeably **better, more fluent** answers, but it costs money and needs an API key.

Having both shows a real production pattern: a **free, fully-local default** that works for anyone, plus a **one-click upgrade** to a stronger cloud model when quality matters. Thanks to Spring AI, flipping the toggle changes only which provider answers — the rest of the system is untouched.

Embeddings: nomic-embed-text vs text-embedding-3-small

There is an important subtlety. The toggle above only changes the **chat / summary** model. The **embeddings always stay on Ollama's `nomic-embed-text` model (768 numbers)**, even when chat is switched to OpenAI. Why? Because the entire vector index is built with 768-dim vectors; switching the embedding model would change the vector size and **force a full, slow re-index** of every chunk. Keeping embeddings fixed means the toggle is **instant and free**.

`text-embedding-3-small` is OpenAI's small, fast, inexpensive embedding model (it produces 1536 numbers). It was the project's original plan and is still wired in as a switch-back option. It is an excellent, low-cost embedding model — but because adopting it would require re-indexing the corpus at 1536 dimensions, CryptoCopilot deliberately keeps embeddings on the **free, local** `nomic-embed-text` so that the whole retrieval layer costs **≈ €0** and never needs a rebuild when you change the chat provider.

> In short: chat/summary = Ollama llama3.2:3b OR OpenAI gpt-4o-mini (your choice, instant). Embeddings/search = always Ollama nomic-embed-text, 768-dim (so no re-index is ever needed).

7. The Analyst Aggregator

The **Analyst** is the component that fuses the four perspectives into **one** opinion. It is **deterministic** — given the same inputs it always produces the same verdict — which makes it auditable. Each input is scored on a small **−2 … +2** scale:

- **ML** — UP or DOWN gives ±2 when the calibrated confidence clears a threshold (else ±1); FLAT gives 0.
- **Technical (ta4j)** — BULLISH/BEARISH × STRONG/MODERATE maps to ±2 / ±1.
- **Fundamental health** — IMPROVING / DETERIORATING gives ±1.
- **News sentiment** — net positive / negative over the last 7 days gives ±1.

The four scores are added into a **−6 … +6** total, which becomes:

- **Direction** — LEAN_BULLISH (≥ +3), LEAN_BEARISH (≤ −3), CONFLICTED (inputs disagree in sign), or NEUTRAL.
- **Conviction** — HIGH (|total| ≥ 4), MEDIUM (2–3), or LOW.
- **Agreement score** — how much the four inputs agree (1 = full agreement).

Two-tier fundamental health

Fundamental "health" is computed from the **best available** data, and the source is reported openly as `healthSource`:

- **Tier 1 — on-chain** (real daily series): rising 7-day averages of active addresses and transfer volume → IMPROVING. In practice **only BTC** has this rich daily series.
- **Tier 2 — CoinGecko**: for the other coins, a rule over 7-day momentum, developer activity, and 24h market-cap change.
- **Tier 3 — unknown**: if neither is available, the coin is simply marked UNKNOWN — never guessed.

The hallucination guard

The Analyst's readable summary is **phrased by the LLM**, but every number in that summary **must already appear in the inputs**. A **hallucination guard** checks this: if the model invents any number, or the LLM is unavailable, the system **falls back to a deterministic template** built from the real numbers. So the Analyst is reliable **even with the LLM switched off** — it simply phrases the same facts more plainly.

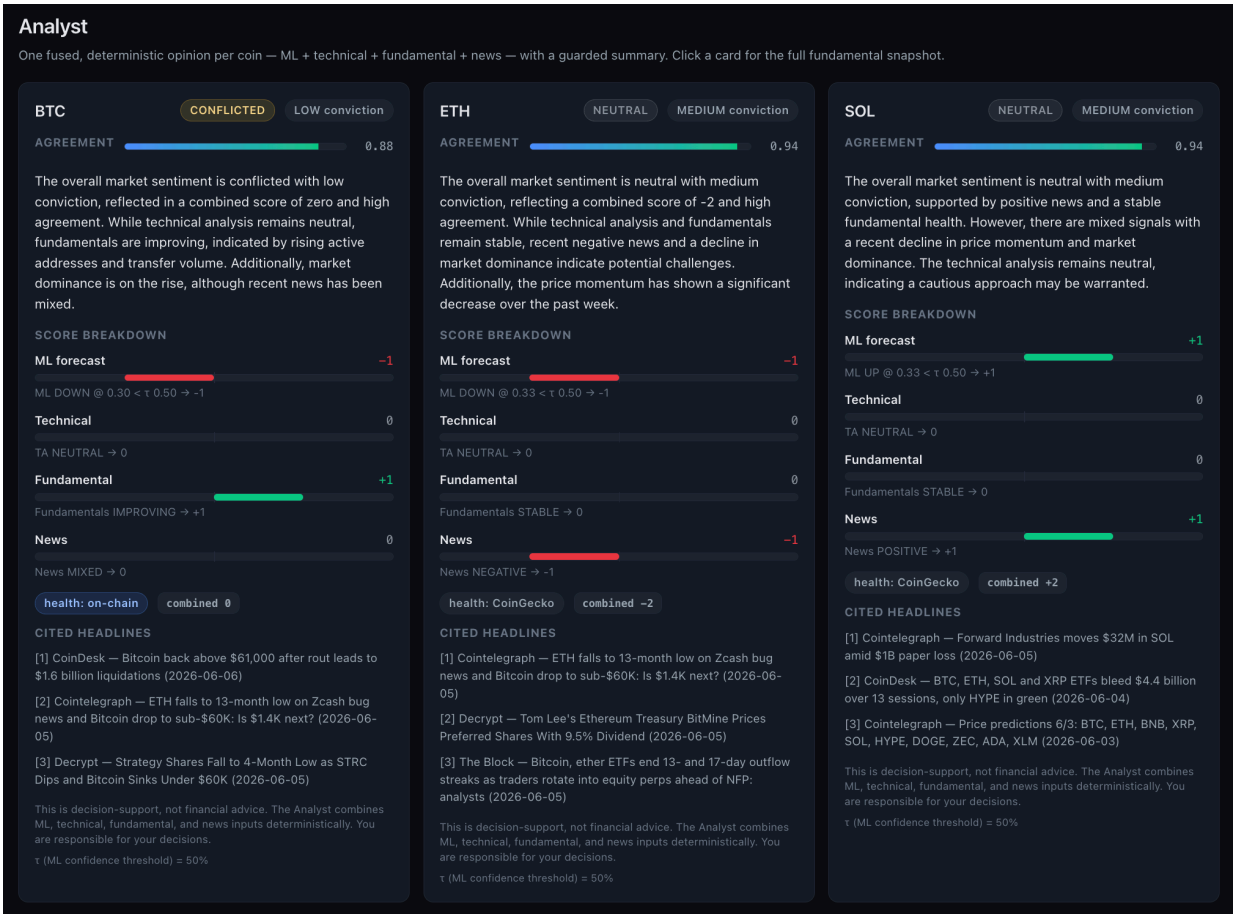

Figure 6 — the Analyst page for three coins. Each shows the direction (e.g. CONFLICTED / NEUTRAL), the agreement score, a plain-language summary, the score breakdown across ML / Technical / Fundamental / News, the health source, and cited headlines.

8. The Paper-Trading Engine

"**Paper trading**" means practising with **fake money** — the same mechanics as real trading, but nothing is ever at risk. This is a hard rule of the project: **no real money, ever; long-only; no shorts; no leverage.** The engine lets the user place orders and then tracks the account.

How a fill works

The "fill" logic (how an order becomes a completed trade) is written once and shared with the backtest, so simulation and live behaviour agree:

- **MARKET order** — fills at the next 1-hour candle's open price, moved by a realistic **0.05% slippage** (a buyer pays slightly more, a seller gets slightly less).

- **LIMIT order** — fills only if a later candle's price range actually reaches the limit price; otherwise it stays **PENDING**.
- A **0.1% taker fee** is charged on every fill (just like a real exchange).

The engine writes four tables — `orders`, `trades`, `positions`, and `account_state` — and takes a clean equity snapshot after each fill, so the equity curve never records a half-applied trade. It also enforces the rules: selling more than you hold is rejected (long-only).

Performance metrics

The **Performance** page marks the account to market and computes standard risk/return metrics over the equity curve: **Sharpe** and **Sortino** ratios (return per unit of risk), **maximum drawdown** (the worst peak-to-trough fall), **win rate**, average win/loss, total trades, and total fees. (See the glossary for each.)

The backtest — an honest result

A **backtest** replays a strategy over real history. The spec's default strategy ("ML-confirmed-by-TA") makes **0 trades**, for an honest reason: the ML service stores only the **latest** forecast per coin, not a historical series, so there is no past ML signal to drive the rule bar-by-bar — and in this calm market no coin is a confident "UP." A reconstructable **TA-only** proxy *does* trade (206 trades) and lands at **Sharpe −1.20** — an honestly negative result, because fees plus a choppy market grind down a naïve trend strategy. The point of this stage is a **correct, well-tested trading engine**, not a profitable strategy.

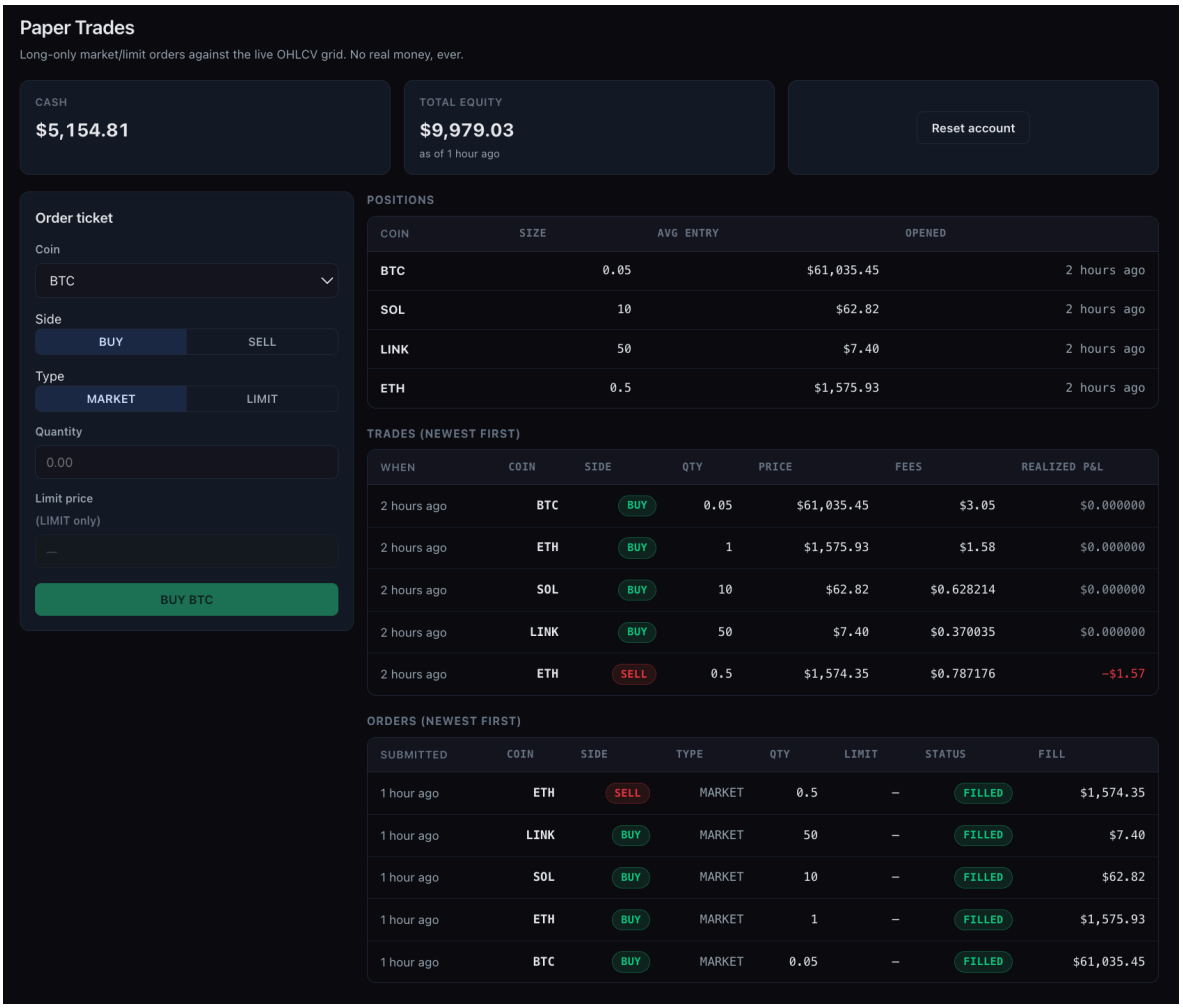

Figure 7 — the Paper Trades page: cash and total equity, the order ticket (BUY/SELL · MARKET/LIMIT), and the positions / trades / orders tables.

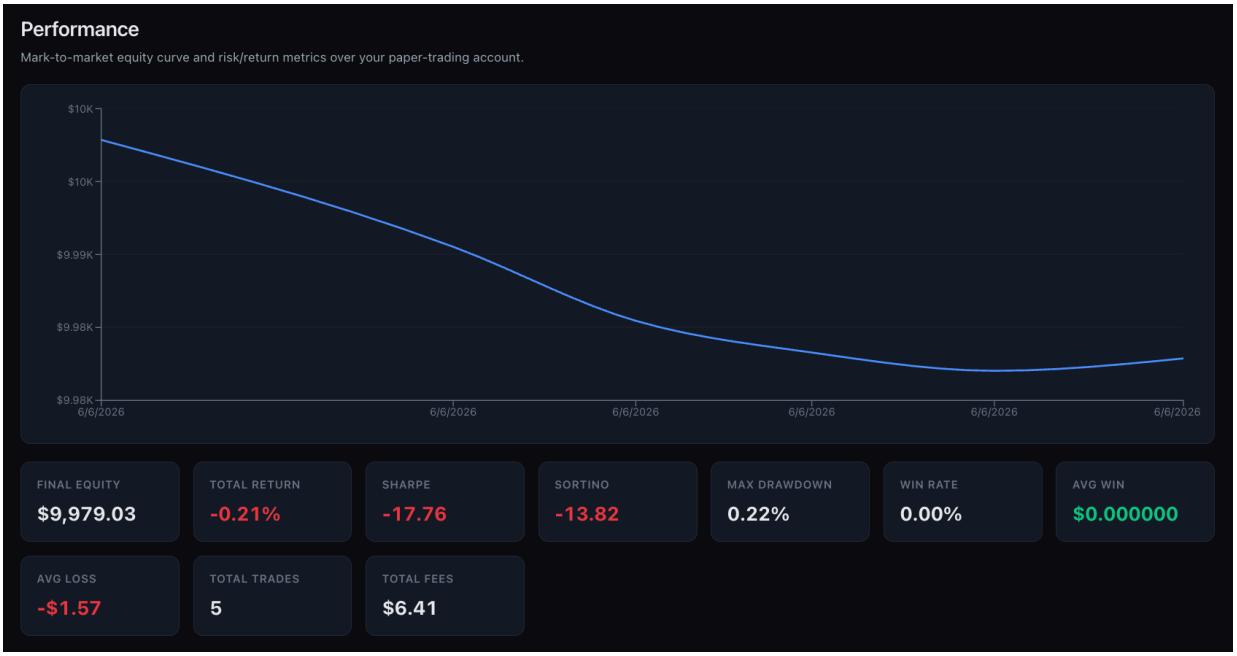

Figure 8 — the Performance page: the mark-to-market equity curve and the risk/return metrics (Sharpe, Sortino, max drawdown, win rate, fees).

9. The Frontend (React)

The frontend is the website you interact with. It is a **thin client**: it holds **no business logic** of its own — it simply displays what the backend returns and submits paper orders.

Technology choices

- **React** — the most popular library for building interactive user interfaces from reusable components.
- **Vite** — a very fast build tool and development server.
- **TypeScript** — JavaScript with types, which catches a whole class of bugs before the code ever runs and mirrors the backend's data shapes exactly.
- **nginx** — a fast web server that serves the built app **and** reverse-proxies `/api` to the backend, so the browser only ever talks to one origin — which means **no CORS configuration is needed**.

Charts

Two charting libraries are used: **klinecharts** draws the professional candlestick charts (with the Ichimoku overlay and 1h/4h/1d switch) on the coin-detail page, and **Recharts** draws the equity curve on the Performance page. (The project first used **TradingView Lightweight Charts** for candles and later swapped to **klinecharts** for its richer built-in indicator overlays.)

The pages

The app has six main pages — **Markets, Signals, Analyst, Researcher (chat), Paper Trades, Performance** — plus a coin-detail page and an **ML Pipeline** page (where you can trigger ingest / train / predict and watch them run). A persistent **"decision-support, not financial advice"** disclaimer sits on every page.

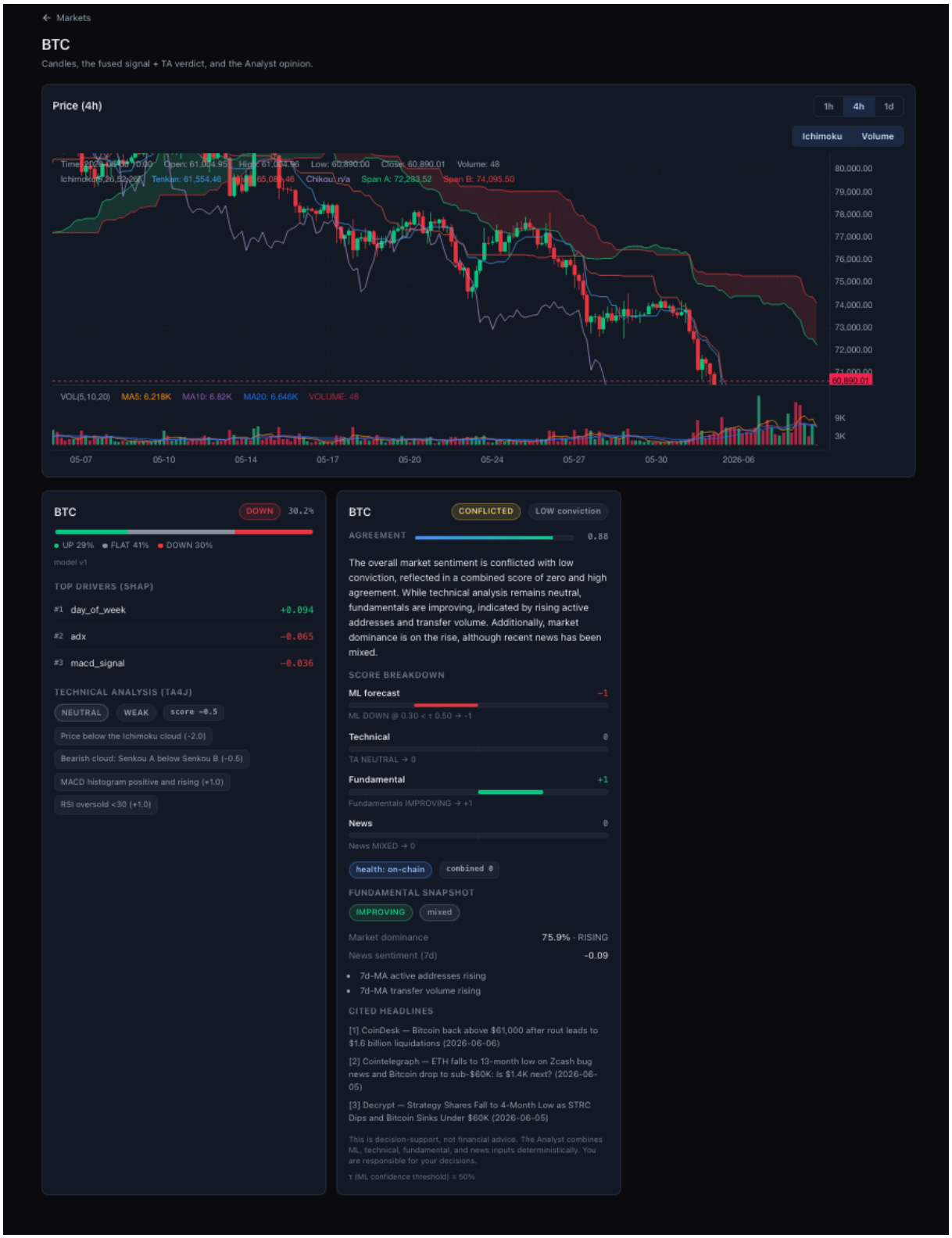

Figure 9 — the coin-detail page: a klinecharts candlestick chart with the Ichimoku overlay, beside the inline ML signal and the full Analyst opinion for that coin.

10. On-Demand Pipeline: the ML API (Python / FastAPI)

Originally the ML jobs only ran on a schedule. The **ml-api** container adds a thin **FastAPI** web layer over the *same* ingest / train / predict code, so those jobs can be launched **on demand** — from the backend (`/api/ml/*`) and the **ML Pipeline** page in the UI. They run as background jobs you can poll for status, and because both the scheduler and the API **share the same model folder**, a model trained from the UI is exactly the one the next prediction uses. FastAPI was chosen because it is the standard, lightweight way to expose Python functions as a fast web API, with automatic docs of its own.

11. Glossary — Key Concepts in Plain Words

Every important term in this project, briefly defined.

Crypto & markets

- **Decentralization** — no single authority (like a central bank) is in control; the network is run by many independent participants.
- **Consensus (PoW / PoS)** — how a decentralized network agrees on the truth. **Proof-of-Work** (Bitcoin) spends energy; **Proof-of-Stake** (Ethereum, Solana…) stakes coins as collateral.
- **Tokenomics** — a coin's economics: total supply, how new coins are created, and incentives.
- **Market cap** — price × circulating supply; a measure of total size.
- **On-chain metric** — activity recorded directly on the blockchain (e.g. active addresses, transfer volume).
- **OHLCV / candle** — Open, High, Low, Close price and Volume for a time period.

Machine learning

- **Feature** — an input number describing the current state (e.g. RSI, recent return).
- **Classification** — predicting a category (here: UP / FLAT / DOWN).
- **Overfitting** — memorizing the training data instead of learning a general pattern; regularization fights it.
- **Gradient boosting / XGBoost** — many small decision trees built in sequence, each fixing the last one's errors.
- **Calibration / isotonic** — adjusting probabilities so "70%" really means 70%.
- **SHAP / Shapley value** — a fair share-out of credit among features, explaining each prediction.
- **ROC-AUC** — ranking skill (0.5 = random, 1.0 = perfect). **Brier** — probability accuracy (lower is better). **F1** — balance of precision and recall.
- **Leakage / embargo** — accidentally using future information; a time gap between splits prevents it.

AI / RAG

- **LLM** — Large Language Model, the kind of AI that generates text.
- **Embedding / vector** — text turned into a list of numbers that captures meaning.
- **Cosine similarity** — how close two vectors (meanings) are.
- **RAG** — Retrieval-Augmented Generation: fetch real sources, then answer using only them.
- **Hallucination** — when an LLM confidently states something false; grounding + the guard prevent it here.
- **VADER sentiment** — a fast rule-based tool that scores text as positive / negative / neutral, used to label news.

Trading

- **Paper trading** — practice trading with fake money.
- **Slippage** — the small price difference between expected and actual fill.
- **Taker fee** — the exchange fee paid when your order fills immediately.
- **Sharpe / Sortino** — return earned per unit of risk (Sortino counts only downside risk).
- **Drawdown** — how far the account fell from its previous peak.

12. Running It & Honest Results

How to run the whole thing

One command brings the system up and fills it with data:

```
cp -n .env.example .env      # add the two free API keys
make demo                    # up + ingest + train + predict + reindex + seed trades
```

Then open **http://localhost:3000** (the app), **:8080/swagger-ui.html** (API docs), and **:8000/docs** (the ML API). The Researcher chat and the LLM-phrased Analyst summary use a local **Ollama**; with Ollama **off**, the chat refuses cleanly and the Analyst uses its deterministic template — every other page is fully populated either way.

Honest results, in one table

Layer	Result	Honest reading
ML direction	macro F1 0.375 · AUC 0.578 · Brier 0.606	A small but real edge; AUC + Brier pass; F1 is a data-limited ceiling (~2y history).
RAG retrieval	recall@8 0.90 · 100% citations	Strong grounded retrieval; refuses out-of-corpus questions and trading advice. ≈ €0 (local).
Paper-trading backtest	default 0 trades · TA proxy Sharpe −1.20	The engine is correct and tested; the strategy is honestly unprofitable in this regime.

What this project demonstrates: a clean polyglot architecture (two languages cooperating through one database), a complete, calibrated, explainable ML pipeline, a production-style Spring Boot backend with technical analysis and a grounded RAG chat, a deterministic decision-fusion Analyst with a hallucination guard, a safe paper-trading engine, and a polished, typed React frontend — all packaged with Docker, tests, and continuous integration.

> ⚠️ Final reminder: CryptoCopilot is a personal learning and portfolio project. It is decision-support, not financial advice. Paper trading only — no real money, ever.